



NINE REALMS

九 州 圣 兵

Master Handoff & Technical Bible

A TFT-style auto-battler · Three.js / WebGL · Chinese mythic-cute
9 weapon-Pandas & 9 magic-Loong (the Nine Sons of the Dragon)

Engineering + design handoff for **Claude Code** / **Codex** / **ChatGPT**
The full Content Bible follows as the next message.

June 21, 2026

Master Handoff & Technical Bible

For Claude Code / Codex / ChatGPT — everything needed to continue the project

Single-player, TFT-style auto-battler. Chinese mythic-cute (xianxia) theme. Three.js / WebGL. Roster: 9 weapon-Pandas + 9 magic-Loong (the Nine Sons of the Dragon).

How to use this document. This is the **engineering + design handoff**. It tells a fresh agent the project vision, exactly what already exists, the environment quirks that bit us, the **recommended production architecture** for Claude Code, concrete **data schemas**, formulas, and a milestone build order. The **full Content Bible** (the complete game-design fill, roster lore, all systems) is delivered as the **next message** right after this PDF — treat that as the canonical *design* source of truth, and treat this PDF as the canonical *engineering* source of truth. Where they overlap, design intent wins; where this PDF gives schemas/formulas/structure, follow it.

Contents

NINE REALMS · 九州圣兵

Master Handoff & Technical Bible

For Claude Code / Codex / ChatGPT — everything needed to continue the project

Contents

PART A — ORIENTATION

1. Project snapshot (status board)

2. Vision (one page)

PART B — WHAT ALREADY EXISTS (the prototype)

3. Prototype architecture

4. Game systems implemented (where to look in main.js)

5. Asset optimization pipeline (reusable — important)

6. The texture/blob problem and the fix (read before touching model loading)

7. Environment constraints we hit (and which go away in Claude Code)

PART C — TARGET ARCHITECTURE (for Claude Code)

8. Why restructure, and what to keep

9. Recommended stack

10. Project structure

11. Core data schemas (TypeScript — implement these exactly)

12. Constants & formulas (single source of truth)

13. Asset loading in the real project (no blob hacks)

14. Testing & verification (do this early — we were flying blind without it)

PART D — DESIGN SYSTEM (condensed; full detail in the Content Bible next message)

15. The Bloodline × Element grid (the core)

16. The 18-unit roster (1★ baselines; ids for code)

Pandas (own GLB per weapon)

Loong (the Nine Sons; weaponless, magic only; placeholder model for now)

17. The 7 traits

18. Economy / shop / leveling (tables already in § 12 constants)

19. Combat & status (formulas in § 12)

20. Items / Edicts / PvE / AI / HP

21. Art & audio direction

PART E — EXECUTION

22. Build order (milestones / tickets)

23. Coding conventions

24. Glossary

25. Appendix — deliverables & commands

Final note to the next agent

PART A — ORIENTATION

1. Project snapshot (status board)

Area	Status	Notes
Core loop (plan→shop→combat→result→income)	Working in prototype	single-file Three.js build
Hex board 7×8, placement, selection	Working	click-to-select, click-to-place, swap
Economy (gold, interest, streak, reroll, XP, level odds)	Working	tunable constants
Shop / bench / 3★ combine	Working	shared-pool not yet modeled
Traits panel	Working (old 3-trait test)	must be replaced by the 7-trait system (this doc)
Combat (move/attack/abilities/crit/damage numbers/HP bars)	Working	5 test units only
Shanghai skyline + dusk sky environment	Working	procedural, not an asset download
Panda model load (93 MB → ~0.8 MB)	Working	geometry-only + separate textures (see § 6, § 7)
Texture display in sandbox	Degraded/uncertain	sandbox blocks <code>blob:</code> ; textures applied in background w/ timeout
Roster = 18 units	Not started	data only designed; see § 16
7-trait engine (Bloodline × Element)	Not started	designed; see § 17
Items / Edicts / PvE bosses / ghost-board AI	Not started	designed; see § 20
Loong models	Not available	use stylized placeholders until modeled
Automated tests	None	add Playwright smoke tests in Claude Code (§ 14)

One-line state: *a fully playable 5-unit vertical slice exists as a single self-contained HTML file; the next job is to lift it into a real Vite+TS+Three.js project and implement the 18-unit roster and 7-trait system.*

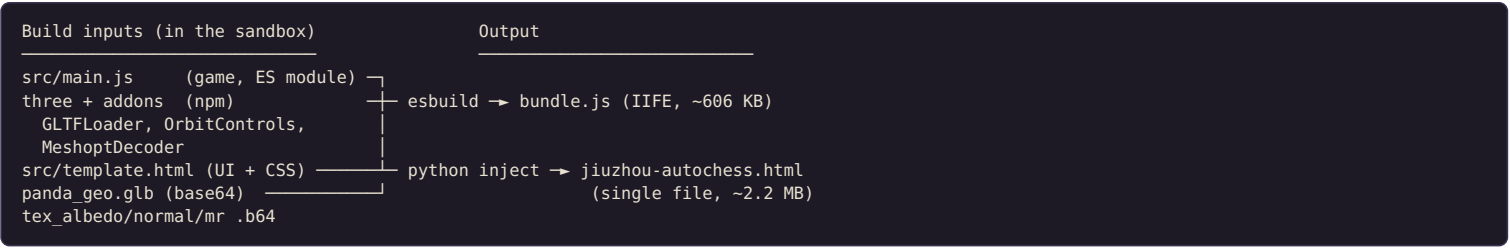
2. Vision (one page)

- **Fantasy:** you are an Immortal Strategist commanding two sacred bloodlines — martial **Pandas** (eighteen weapons) and arcane **Loong** (the Nine Sons, wielders of the Five Elements) — climbing the Nine Realms to defeat the Heavenly General.
- **Pillars:** (1) readable depth — every fight explains itself; (2) two bloodlines, one board — the martial/arcane join is the strategic core; (3) mythic-cute premium feel.
- **The core design idea (read this):** units are tagged on **two axes** — **Bloodline** (Panda/Loong) and **Element** (金木水火土). Elements are shared by both species, so a Fire panda and a Fire loong activate the same trait. That shared axis is the mechanic that makes the two halves want to share a board. Total **7 traits** (2 bloodline + 5 element), not 18. A signature deep mechanic — the **Wuxing 生克 cycle** — bolts on later with zero roster changes. Full rationale in the Content Bible § 0/§ 12.

PART B — WHAT ALREADY EXISTS (the prototype)

3. Prototype architecture

The current deliverable is **one self-contained .html file** (~2.2 MB) that runs offline with no network calls.



- **Rendering:** Three.js r160, `ACESFilmicToneMapping`, sRGB, PCFSoft shadows, `FogExp2`, custom gradient sky shader, hemispheric + directional (sun) + rim lights.
- **Bundling:** everything (Three + loaders + game) compiled by **esbuild** into one IIFE so the file is dependency-free. Meshopt was chosen over Draco because the **MeshoptDecoder embeds its own WASM** (no external decoder file needed).
- **UI:** plain HTML/CSS overlay (no framework). Chinese game styling with Google Fonts (Noto Serif SC / ZCOOL XiaoWei / Cinzel). Floating HP/mana bars and damage numbers are **projected DOM elements** (world→screen each frame), not sprites.
- **Input:** raycast against hex meshes for hover/place; click-to-select/place/swap; keys D (reroll), F (buy XP), Space (begin combat), E (sell).

4. Game systems implemented (where to look in main.js)

System	Implementation summary
Board	<code>hexWorld(col, row)</code> , 7×8 grid, bottom 4 rows = player; <code>hexMeshes[]</code> carry <code>{col, row, owner, occupant}</code>
Units	<code>class Unit</code> clones the panda pivot, per-unit cloned material (tint), ground/team rings, star sprites, combat state
Shop	<code>rollShop()</code> uses per-level odds; <code>buy()</code> , <code>reroll()</code>
Combine	<code>checkCombine()</code> merges 3 same-key/star → star-up (bench+board counted)
Economy	<code>nextRound()</code> income = 5 + interest(≤5) + streak; XP +2/round; <code>buyXP()</code>
Traits (OLD)	<code>recalcTraits()</code> counts 3 placeholder traits — replace with \$17
Combat	<code>stepCombat(dt)</code> steering + separation, attack on <code>1/AS</code> , mana, abilities, crit 25%/×1.8; <code>castAbility()</code>
Enemy	<code>genEnemyTeam()</code> scales count/star/stats by round; mirrored placement
Result	<code>endCombat()</code> applies HP deltas; <code>doGameOver()</code> ; revive between rounds
FX	<code>flash()</code> , <code>aoeRing()</code> , projected damage numbers + HP/mana bars

5. Asset optimization pipeline (reusable — important)

The uploaded panda was **93.66 MB** (1.5 M triangles, three 4096² PBR textures). Pipeline that brought it to **~0.84 MB** while staying recognizable:

```
# tools: gltf-transform CLI (npm), pygltflib (pip)
gltf-transform weld      in.glb weld.glb
gltf-transform simplify weld.glb simp.glb --ratio 0.025 --error 0.0015 # ~47k tris
gltf-transform resize   simp.glb simp.glb --width 1024 --height 1024
gltf-transform webp      simp.glb simp.glb --quality 78
gltf-transform meshopt   simp.glb panda_game.glb # ~0.84 MB
```

Use the **same recipe for the 9 real panda GLBs** and future Loong models. Target ≈ 30–50 k tris and 1024² WebP per unit so a 16-unit board stays light.

6. The texture/blob problem and the fix (read before touching model loading)

Symptom in the sandbox: `THREE.GLTFLoader: Couldn't load texture blob:...`, then the loader hung. **Cause:** the artifact iframe **blocks blob: URL fetches**; GLTFLoader turns embedded textures into blob URLs to load them.

Fix used in the prototype (two parts): 1. Ship the model as **geometry-only** (`panda_geo.glb`): the GLB's JSON header was edited to remove `images/textures/samplers` and the material's texture refs, **leaving the binary/meshopt chunk byte-identical**. With no images, GLTFLoader does zero image work and can never stall. 2. Ship the **3 textures separately** (base64 WebP) and apply them **in the background after the game starts**, decoded via `createImageBitmap(blob)` (decodes bytes directly — no URL) with a **6 s timeout** so the game always runs even if a texture stalls.

```
# geometry-only GLB: edit only the JSON chunk, keep BIN chunk identical
# (removes images/textures/samplers + material tex refs; keeps EXT_meshopt_compression)
```

In Claude Code this whole problem disappears when assets are served over `http://` by a dev server (Vite). You can keep textures in the

GLB normally. The geometry-only split was a *sandbox workaround*, not a requirement. See § 13.

7. Environment constraints we hit (and which go away in Claude Code)

Constraint (sandbox / artifact)	Impact	In Claude Code
blob: URL fetch blocked	broke GLTF texture load	Gone (served over http)
No headless browser available	couldn't auto-test render	Use Playwright to screenshot/test
Network allowlist (npm/pypi/github only)	couldn't fetch asset marketplaces	Normal network; still prefer local assets
Single-file artifact, no localStorage	no persistence, no modules at runtime	Real project: Vite, ES modules, IndexedDB if needed
Google Fonts via CDN <link>	ok in artifact	Self-host fonts for offline/perf

PART C — TARGET ARCHITECTURE (for Claude Code)

8. Why restructure, and what to keep

Keep: the *game logic* and *math* (they're proven), the *environment look*, the *asset pipeline*. Change: move from one giant HTML file to a **Vite** + **TypeScript** + **Three.js** project with real modules, typed data, and tests. This makes the 18-unit roster, 7-trait engine, items, and PvE tractable and testable.

9. Recommended stack

- **Vite** (dev server + build) · **TypeScript** (typed data is the whole point with this much content) · **Three.js** (latest) · **Zustand** or a tiny event bus for game state · **Playwright** for smoke tests.
- No heavy UI framework needed; the HUD can stay DOM+CSS (optionally React if the team prefers). Keep WebGL and DOM-HUD decoupled.

10. Project structure

```
nine-realms/
index.html
vite.config.ts
public/
  models/      panda_*.glb, loong_*.glb      # optimized via $5 recipe
  textures/    *.webp
  fonts/       NotoSerifSC, ZCOOLXiaoWei, Cinzel (self-hosted)
  audio/       sfx/*.ogg, music/*.ogg
src/
  main.ts
  engine/
    renderer.ts scene.ts camera.ts lights.ts sky.ts loop.ts
    assets.ts    # GLB/texture loading (normal http; no blob hacks)
    fx.ts        # flashes, aoe rings, projected DOM HP bars + dmg numbers
  world/
    board.ts     # hex grid, hexWorld(), neighbors, placement
    environment.ts # Shanghai skyline, arena, clouds (port from prototype)
  data/
    elements.ts  bloodlines.ts roles.ts
    units.ts     # the 18 UnitDef (see §16, §schema)
    traits.ts    # 7 TraitDef (see §17)
    items.ts     edicts.ts enemyBoards.ts shopOdds.ts levelXp.ts
  systems/
    economy.ts  shop.ts combine.ts traits.ts(apply) combat.ts
    abilities.ts targeting.ts status.ts enemyAI.ts rewards.ts
  state/
    gameState.ts # store + phase machine
  ui/
    Hud.tsx/ts  Shop.ts Bench.ts TraitPanel.ts Tooltip.ts Banners.ts
  tests/
    smoke.spec.ts # boot, load model, place unit, run a fight
```

11. Core data schemas (TypeScript — implement these exactly)

```

export type Bloodline = 'panda' | 'loong';
export type Element   = 'metal' | 'wood' | 'water' | 'fire' | 'earth';
export type Role      = 'vanguard' | 'blade' | 'ranger' | 'phantom' | 'mystic' | 'warden';
export type DamageType = 'physical' | 'magic' | 'true';
export type StatusKind = 'burn' | 'stun' | 'root' | 'armor_break' | 'shield' | 'hot';
export type AbilityKind = 'nuke' | 'aoe' | 'shield' | 'heal_buff' | 'dash' | 'control';
export type TargetMode  = 'nearest' | 'lowest_hp' | 'densest' | 'highest_ad'
                        | 'backline' | 'self' | 'allies';

export interface AbilityDef {
  id: string;
  name: { zh: string; en: string };
  kind: AbilityKind;
  target: TargetMode;
  manaCost: number;
  damage?: number;           // 1* base, scaled by STAR_MULT and Water/Loong AP buffs
  damageType?: DamageType;
  radius?: number;           // world units for aoe
  durationMs?: number;       // stun/root/burn length
  status?: StatusKind;
  shield?: number;
  heal?: number;
  notes?: string;           // human-readable for tooltip
}

export interface UnitDef {
  id: string;                // 'p_goldchain', 'l_suanni'
  name: { zh: string; en: string };
  bloodline: Bloodline;
  element: Element;
  role: Role;
  cost: 1 | 2 | 3 | 4 | 5;
  stats: { hp:number; ad:number; as:number; armor:number;
           range:number; manaStart:number; manaMax:number };
  ability: AbilityDef;
  model: { glb: string; scale?: number; tint?: number }; // loong: placeholder glb until modeled
  art?: string;           // concept art filename
}

export interface TraitTier { count:number; label:{zh:string;en:string}; effect:string }
export interface TraitDef {
  id: string;                // 'panda','loong','metal',...
  axis: 'bloodline' | 'element';
  name: { zh:string; en:string };
  color: string;             // UI hex
  tiers: TraitTier[];        // ascending count thresholds
  // apply(activeTier, units, combatCtx) implemented in systems/traits.ts
}

export interface ItemDef { id:string; name:{zh:string;en:string}; effect:string; stat?:Partial<UnitDef['stats']>; status?:StatusKind }
export interface EdictDef { id:string; name:{zh:string;en:string}; tier:'jade' | 'gold' | 'prism';
category:'combat' | 'economy' | 'trait' | 'item' | 'risk'; effect:string }
export interface EnemyBoardDef {
  round:number; theme: Element | 'mixed';
  units: { unitId:string; star:1|2|3; hex:[number,number] }[];
  boss?: 'stone_warlord' | 'heavenly_general';
}

export interface UnitInstance {           // runtime
  uid:number; def:UnitDef; star:1|2|3; location:'bench' | 'board';
  hex?:[number,number];
  chp:number; mana:number; shield:number; alive:boolean;
  statuses: { kind:StatusKind; until:number; data?:number }[];
}

export interface GameState {
  phase:'planning' | 'combat' | 'result' | 'gameover';
  round:number; gold:number; level:number; xp:number;
  playerHp:number; enemyHp:number;
  bench:UnitInstance[]; board:UnitInstance[]; shop:(string|null)[];
  shopLocked:boolean; winStreak:number; lossStreak:number;
  lastResult:'win' | 'loss' | null; selected?:UnitInstance;
}

```

12. Constants & formulas (single source of truth)

```

export const STAR_MULT = { 1: 1.0, 2: 1.8, 3: 3.2 };
export const BENCH_CAP = 9;
export const boardCap = (level:number) => level;           // units allowed on board

// combat
export const physMitigation = (dmg:number, armor:number) => dmg * 100 / (100 + armor);
export const CRIT_CHANCE = 0.25, CRIT_MULT = 1.8;
export const MANA_ON_ATTACK = 10;                        // + small on being hit
export const MOVE_SPEED = 2.6;                           // world units / s
export const COMBAT_CAP_S = 30;

// economy
export const income = (gold:number, streak:number) =>
  5 + Math.min(5, Math.floor(gold/10)) + Math.min(3, Math.max(0, streak-1));
export const sellRefund = (cost:number, star:number) => cost * Math.pow(3, star-1);
export const REROLL_COST = 2, BUYXP_COST = 4, BUYXP_GAIN = 4, XP_PER_ROUND = 2;

// player HP
export const lossDamage = (round:number, survivingEnemies:number) => 2 + round + survivingEnemies*2;
export const winDamage = (round:number, survivingAllies:number) => Math.round(3 + round*0.8 + survivingAllies*1.5);

// shop odds: P(cost) per slot by player level
export const SHOP_ODDS: Record<number, [number,number,number,number,number]> = {
  2:[75,25,0,0,0], 3:[60,30,10,0,0], 4:[45,33,20,2,0], 5:[30,40,25,5,0],
  6:[22,35,30,10,3], 7:[16,30,33,17,4], 8:[10,22,32,26,10], 9:[5,15,30,30,20],
};
export const LEVEL_XP: Record<number, number> =
  { 2:0, 3:6, 4:10, 5:20, 6:36, 7:48, 8:66, 9:84 }; // XP to reach next level
export const POOL_COPIES = { 1:18, 2:15, 3:12, 4:9, 5:7 }; // shared pool per unit

```

13. Asset loading in the real project (no blob hacks)

- Put optimized `.glb` in `public/models/`. Load with `GLTFLoader` + `setMeshoptDecoder(MeshoptDecoder)` over `http://` — textures embedded in the GLB are fine because the dev server serves them as normal URLs.
- Keep the **\$ 5 optimization recipe** as an `npm run assets` script (gltf-transform) so re-importing the 9 panda models is one command.
- Loong have **no models yet** — implement a `loongPlaceholder()` (coiled tube + element-tinted emissive + particle aura) and swap to real GLBs later behind the same `UnitDef.model` field.

14. Testing & verification (do this early — we were flying blind without it)

- Playwright** smoke test: boot the app, wait for "loaded", assert a panda mesh exists (`scene.getObjectByName`), place a unit, start combat, assert HP bars render and a winner is decided. Screenshot to `tests/__shots__`.
- Add a tiny **debug panel** (toggle with `~`) that can: spawn any unit/star, force an element/bloodline tier, skip to a round, and dump `GameState`. This makes design tuning and bug-repro fast.
- Validate the model visually in CI by rendering one frame to PNG (headless WebGL via `playwright` + real Chromium — available in Claude Code, unlike the sandbox).

PART D — DESIGN SYSTEM (condensed; full detail in the Content Bible next message)

15. The Bloodline × Element grid (the core)

	金 Metal	木 Wood	水 Water	火 Fire	土 Earth
 Panda	Goldchain(2), Dragoncleaver(4)	Jade Spear(1), Bamboo Sage(1)	Bluetide Saber(2), Twilight Fang(4)	Embershot(3), Crimson Glaive(5)	Magma Maul(3)
 Loong	Pulao(2), Yazi(4)	Qiuniu(2)	Fuxi(3), Chiwen(5)	Chaofeng(3), Suanni(5)	Bi'an(1), Bixi(4)

Read a **column** = element comp (mixes species). Read a **row** = bloodline comp (one species). Best boards do both.

16. The 18-unit roster (1★ baselines; ids for code)

Pandas (own GLB per weapon)

id	name 中文/EN	cost	element	role	hp	ad	as	armor	rng	mana	ability
p_spear	翠玉枪 Jade Spear	1	wood	vanguard	700	45	0.70	40	1	60	Vine Lunge — thrust + root 1s
p_staff	青竹棍僧 Bamboo Sage	1	wood	warden	650	38	0.70	35	1	50	Staff Sweep — knockback+slow, heal lowest ally
p_chain	金链双截 Goldchain	2	metal	phantom	550	55	0.85	25	1	0	Whirl Chain — every 3rd hit chains + Metal shred
p_saber	碧波刀客 Bluetide Saber	2	water	blade	600	58	0.80	28	1	60	Tidal Slash — dash to lowest HP, lifesteal 30%
p_bow	赤羽神射 Embershot	3	fire	ranger	520	60	0.75	18	4	70	Triple Ember — 3 arrows + Burn
p_hammer	熔山战锤 Magma Maul	3	earth	vanguard	950	50	0.60	50	1	80	Quake Slam — AoE stun 1.2s + self shield
p_axe	龙吞巨斧 Dragoncleaver	4	metal	blade	1000	78	0.78	35	1	70	Dragon Cleave — frontal cleave + armor shred 30%
p_daggers	紫渊双刃 Twilight Fang	4	water	phantom	720	70	0.95	22	1	50	Abyss Step — leap to carry, next 3 hits crit
p_glaive	赤焰偃月 Crimson Glaive	5	fire	blade	1050	85	0.80	38	2	80	Whirling Moon — growing spin AoE + Burn

Loong (the Nine Sons; weaponless, magic only; placeholder model for now)

id	name 中文/EN (myth)	cost	element	role	hp	ad	as	armor	rng	mana	ability
L_bian	狴犴 Bi'an (Law tiger)	1	earth	warden	680	30	0.60	38	2	60	Shackle Decree — stun 1.5s + armor break 20%
L_qiuniu	囚牛 Qiuniu (Music)	2	wood	warden	560	25	0.60	24	3	50	Song of Renewal — HoT aura + AS buff
L_pulao	蒲牢 Pulao (Bell roar)	2	metal	mystic	520	28	0.60	20	3	60	Thunder Toll — sonic cone, stun 0.8s
L_chaofeng	嘲凤 Chaofeng (Roof phoenix)	3	fire	mystic	560	30	0.65	22	3	70	Sky Dive — backline burst + knock-up
L_fuxi	负屃 Fuxi (Scholar)	3	water	warden	540	26	0.60	22	4	80	Inkflow Talisman — buff ally AP + mana
L_bixi	赑屃 Bixi (Stele turtle)	4	earth	vanguard	1200	35	0.50	60	1	90	Stone Stele — wall shield + taunt
L_yazi	睚眦 Yazi (Blade fury)	4	metal	phantom	700	65	0.90	26	2	60	Vengeance Edge — 5 seeking spectral blades
L_suanni	狻猊 Suanni (Flame lion)	5	fire	mystic	820	40	0.60	30	3	100	Lotus Inferno — huge fire nova on densest clump
L_chiwen	螭吻 Chiwen (Sea swallower)	5	water	mystic	880	38	0.60	30	4	90	Deluge — row tidal wave; cleanse Burns, douse Fire

17. The 7 traits

Bloodline — 熊猫 Panda (distinct Pandas): **3** +15% AD/+8% AS · **6** +35% AD/+15% AS, attacks cleave 25% · **9** +60% AD/+25% AS, first hit crits, cleave 50%. **Bloodline — 应龙 Loong** (distinct Loong): **3** +20 mana start/+20% AP · **6** +50% AP, abilities echo 25% · **9** +90% AP, first ult double-casts, rolling Elemental Storm.

Elements (distinct units of that element; breakpoints scaled to pool): - **金 Metal** (4): 2 → attacks apply −15% armor · 4 → −30% + 8 mana refund on shred-kills. - **木 Wood** (3): 2 → 2% max HP/s regen · 3 → 4%/s + 15% HP starting shield. - **水 Water** (4): 2 → +25% AP, abilities −10 mana · 4 → +50% AP, first cast free. - **火 Fire** (4): 2 → apply Burn (true DoT) · 4 → Burn stacks & spreads on death. - **土 Earth** (3): 2 → front rows +20 armor & 12% HP shield · 3 → shield refreshes at 50% HP.

Wuxing 生克 (ship later, no roster change): Generating 相生 (Wood→Fire→Earth→Metal→Water→Wood) gives a nourished active element +1 effective tier. Overcoming 相克 (Wood→Earth→Water→Fire→Metal→Wood) → +12% dmg to overcome enemies, −12% from your overcomer. Marketing hook: *"master the cycle."*

18. Economy / shop / leveling (tables already in § 12 constants)

Income 5 + interest(≤5) + streak(0/1/2/3); reroll 2; buy XP 4→4XP, +2/round free; level cap 9, board cap = level; 5 shop slots, freezable; shared pool (POOL_COPIES). Odds & XP curve in § 12.

19. Combat & status (formulas in § 12)

Real-time ticks; nearest-target default with role overrides (Phantoms jump backline; Bixi/Bi'an taunt). Statuses MVP: **Burn, Stun, Root, Armor-break, Shield, HoT**. Crit 25%/×1.8. Two damage types (physical mitigated by armor; magic ignores armor for MVP; true = burns/executes). Fight 12–22 s, 30 s cap.

20. Items / Edicts / PvE / AI / HP

- **Items (MVP 8, no recipe grid yet):** Tiger Tally +20% AD · Jade Talisman +30 AP · Iron Robe +200 HP · Wind Bell +15% AS · Spirit Gourd 200 start-shield · Cinnabar Brand on-hit Burn · Mirror Pendant reflect 15% magic · Dragon Pearl –15 first-cast mana. Later: components→completed, Artifacts, **trait Emblems** (grant a Bloodline/Element to a non-member).
- **Edicts (天赐, augments):** offered R2/5/9, 3 choices, reroll once for 1 gold; tiers Jade/Gold/Prism; 12 starter ideas in Content Bible § 18.
- **PvE/bosses:** PvE at R1/3/6/9/12/15 + final; mid-boss 石魑将 **Stone Warlord** (R9, Earth wall), final 天将·叛龙 **Heavenly General** (~R20, weapon phase → element phase punishing mono-element).
- **AI ghost boards:** scripted per round (count/star/items/element theme), named rivals (Pyre Sect, Stoneguard, Shadowblade, Merchant). Scale $\min(8, 1+\text{floor}(R*0.9))$ units, +6%/round stats.
- **HP:** start 100/100; loss $2+\text{round}+\text{enemies}*2$; win $3+\text{round}*0.8+\text{allies}*1.5$; comeback via loss-streak gold + better reward choices.

21. Art & audio direction

- **Pandas = the visual anchor** (your 9 concept images). Map each to its element color: gold nunchaku & dragon-axe = Metal(gold); red bow & guandao = Fire(red); blue saber & purple daggers = Water(blue/violet); teal spear & bamboo staff = Wood(green); orange lion-hammer = Earth(orange).
- **Loong:** sinuous, weaponless, **element-colored energy**; magic circles (法阵) under casts; calligraphy strokes for Fuxi; floating spectral blades for Yazi. Keep clearly *dragon* + *casting*.
- **VFX vocab:** element-tinted particles, sword-arc trails (Panda), 3★ adds halo + extra trail. **Board** can swap per realm later (bamboo sea / molten forge / jade sea / thunder peak / stone tombs).
- **Audio:** guzheng/erhu/dizi/taiko; per-element cast timbres (Fire crackle, Water wash, Metal ring, Wood block, Earth thud); SFX: buy/sell/reroll/level/round-gong/hit/crit/cast/death/trait-activate/win/loss; 3★ = bell+choir; 9-bloodline = dragon roar.

PART E — EXECUTION

22. Build order (milestones / tickets)

- M0 — Project bootstrap.** Vite+TS+Three scaffold; port renderer/scene/camera/lights/sky/environment from the prototype; load one optimized panda over http (no blob hack); Playwright smoke test boots & sees the mesh.
- M1 — Board & placement.** Port hex board, selection, click-to-place/swap, bench; debug panel (–) to spawn units.
- M2 — Roster data.** Implement the 18 `UnitDef` (§ 16), constants/formulas (§ 12). Pandas use their GLBs; Loong use `loongPlaceholder()`.
- M3 — Trait engine.** Implement the 7 `TraitDef` (§ 17): count distinct per Bloodline & Element, apply tiered buffs at combat start, render all 7 in the trait panel (Bloodlines top, 5 Elements below).
- M4 — Combat & abilities.** Port combat; implement 6 ability templates + 5 statuses; wire each unit's ability; crit/mana/burn; damage numbers & HP/mana bars.
- M5 — Economy & shop.** Shared pool (POOL_COPIES), level odds, freeze, sell refunds, 3★ combine, XP/level.
- M6 — PvE, AI, rewards, Edicts.** Ghost-board schedule, PvE/bosses, item drops + choice, Celestial Edicts at R2/5/9.
- M7 — Polish.** Per-element VFX/SFX, banners, victory/defeat, balance pass using the debug panel; then (post-MVP) Wuxing cycle, component items, emblems, Loong models, per-realm boards.
- Handoff rule (per the bible):** give the agent one milestone at a time, name the goal, point to the source table in this PDF or the Content Bible, and instruct: *"do not invent new traits/units in this ticket; list any missing data at the end."*

23. Coding conventions

- TypeScript strict. Data (`src/data/*`) is **pure declarative** — no logic. Logic lives in `src/systems/*` and reads data. This lets designers tune numbers without touching systems.
- Keep **WebGL** and **DOM HUD** decoupled; communicate via the state store + an event bus (`unit:placed`, `combat:start`, `trait:activated` ...).
- All tunable numbers live in `src/data/constants.ts` (mirror of § 12). No magic numbers in systems.
- Deterministic combat option: seedable RNG for crits/targeting so tests and replays are reproducible.
- Every new unit/trait/item is **data-only**; if it needs new behavior, add a reusable template, not a special case.

24. Glossary

- 熊猫 (xióngmāo) Panda · 应龙/龙 (yìnglóng/lóng) **Loong** Chinese dragon (use "Loong", not "dragon").
- 五行 Wǔxíng Five Elements: 金 Metal, 木 Wood, 水 Water, 火 Fire, 土 Earth.
- 相生 Shēng Generating cycle · 相克 Kè Overcoming cycle.
- 龙生九子 "the dragon has nine sons" — our Loong roster. Standard list (Li Dongyang, 1516): 囚牛 Qiuniu, 睚眦 Yazi, 嘲风 Chaofeng, 蒲牢 Pulao, 狻猊 Suanni, 赑屃 Bixi, 狴犴 Bì'ān, 负屃 Fuxi, 螭吻 Chiwen.
- 天赐 Tiāncì "Celestial Edicts" = our augments. 羁绊 Jībàn = synergies/traits. 升星 = star-up. 开战 = begin combat.
- STAR_MULT {1:1, 2:1.8, 3:3.2}. **Bloodline** = origin/species trait. **Element** = class trait.

25. Appendix — deliverables & commands

Files produced so far (in project outputs): - `jiuzhou-autochess.html` — the playable single-file prototype (5 test units). - `NineRealms_ContentBible.md` — the full design fill (delivered next as the Content Bible). - This handoff PDF. - Asset intermediates (re-creatable): `panda_geo.glb`, `tex_albedo/normal/mr.b64`.

Re-create the optimized panda (or any model):

```
npm i -g @gltf-transform/cli
gltf-transform weld in.glb w.glb && \
gltf-transform simplify w.glb s.glb --ratio 0.025 --error 0.0015 && \
gltf-transform resize s.glb s.glb --width 1024 --height 1024 && \
gltf-transform webp s.glb s.glb --quality 78 && \
gltf-transform meshopt s.glb out.glb
```

Bundle the prototype (legacy, for reference):

```
esbuild src/main.js --bundle --format=iife --minify --target=es2020 --outfile=bundle.js
```

Final note to the next agent

The math and feel are proven; the content is fully designed. Your job is to **lift the proven prototype into a typed, tested project and pour in the 18-unit / 7-trait content**. Build one milestone at a time, keep data declarative, and lean on the debug panel + Playwright so you never fly blind the way the prototype phase did. The Content Bible (next message) is your design source of truth; this PDF is your engineering source of truth.

— End of Master Handoff & Technical Bible —